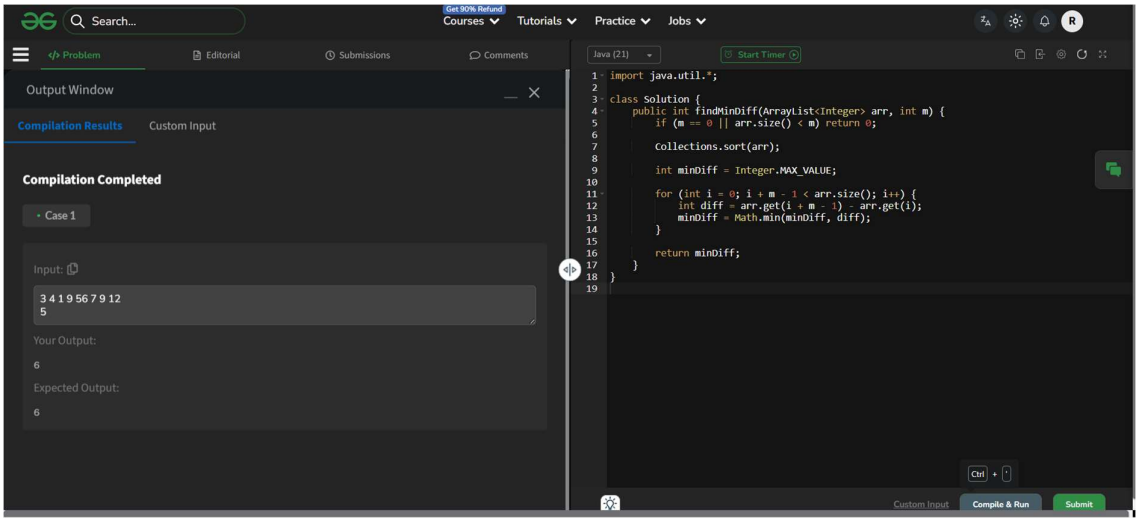
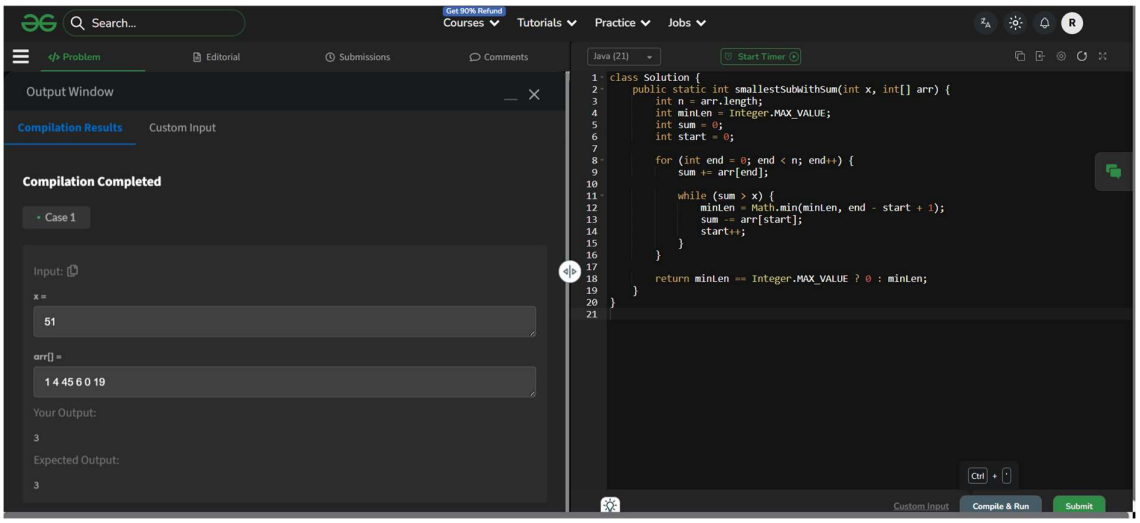
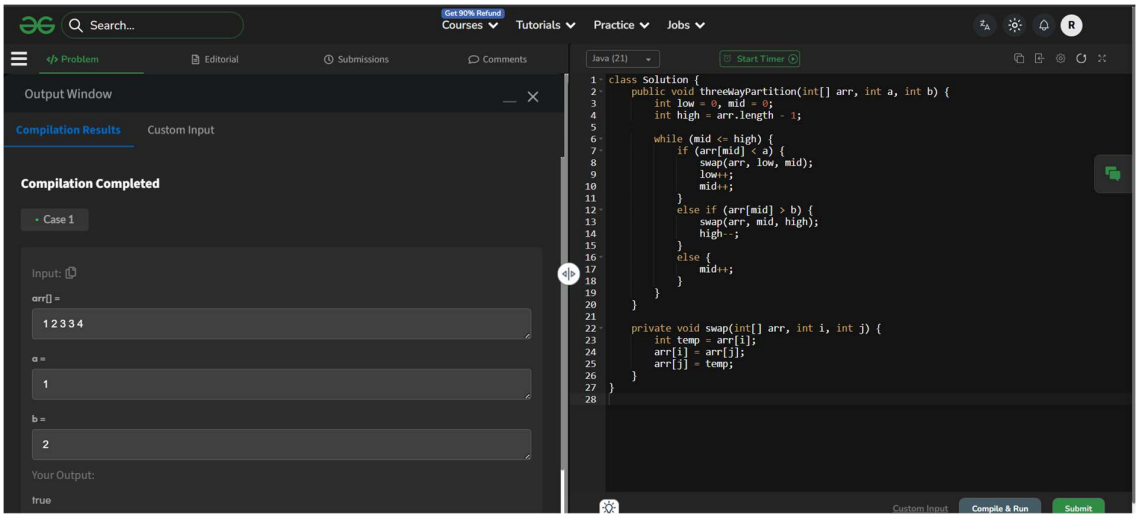
1.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
3 4 1 9 56 7 9 12
5
Your Output:
6
Expected Output:
6
Java (21)
Start Timer
import java.util.*;
class Solution {
public int findMinDiff(ArrayList<Integer> arr, int m) {
if (m == 0 || arr.size() < m) return 0;
Collections.sort(arr);
int minDiff = Integer.MAX_VALUE;
for (int i = 0; i + m - 1 < arr.size(); i++) {
int diff = arr.get(i + m - 1) - arr.get(i);
minDiff = Math.min(minDiff, diff);
}
return minDiff;
}
}
Ctrl + '
Custom Input
Compile & Run
Submit

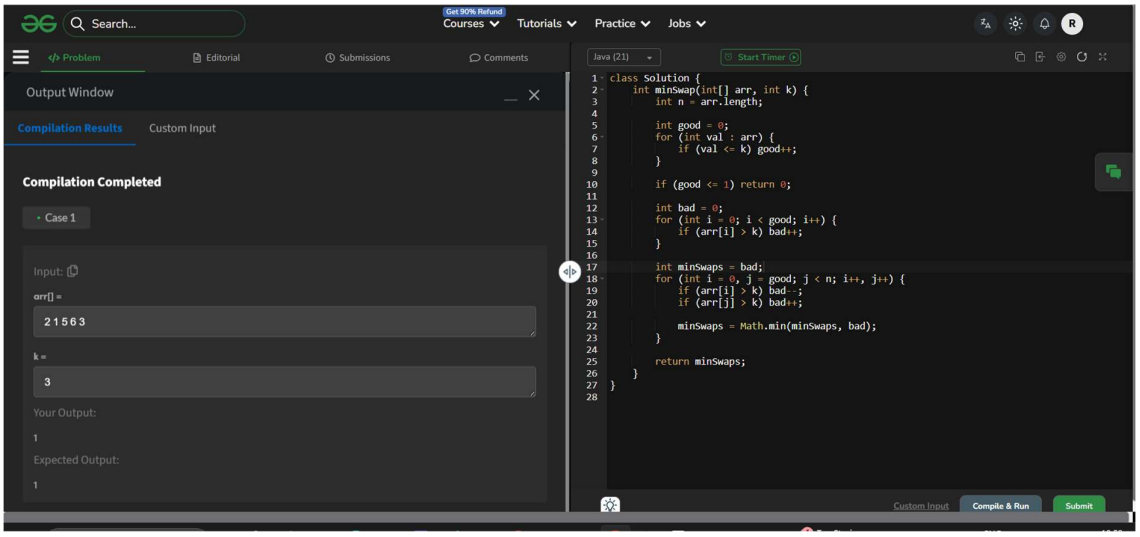
2.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
x =
51
arr[] =
1 4 45 6 0 19
Your Output:
3
Expected Output:
3
Java (21)
Start Timer
class Solution {
public static int smallestSubWithSum(int x, int[] arr) {
int n = arr.length;
int minLen = Integer.MAX_VALUE;
int sum = 0;
int start = 0;
for (int end = 0; end < n; end++) {
sum += arr[end];
while (sum > x) {
minLen = Math.min(minLen, end - start + 1);
sum -= arr[start];
start++;
}
}
return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
}
Ctrl + '
Custom Input
Compile & Run
Submit

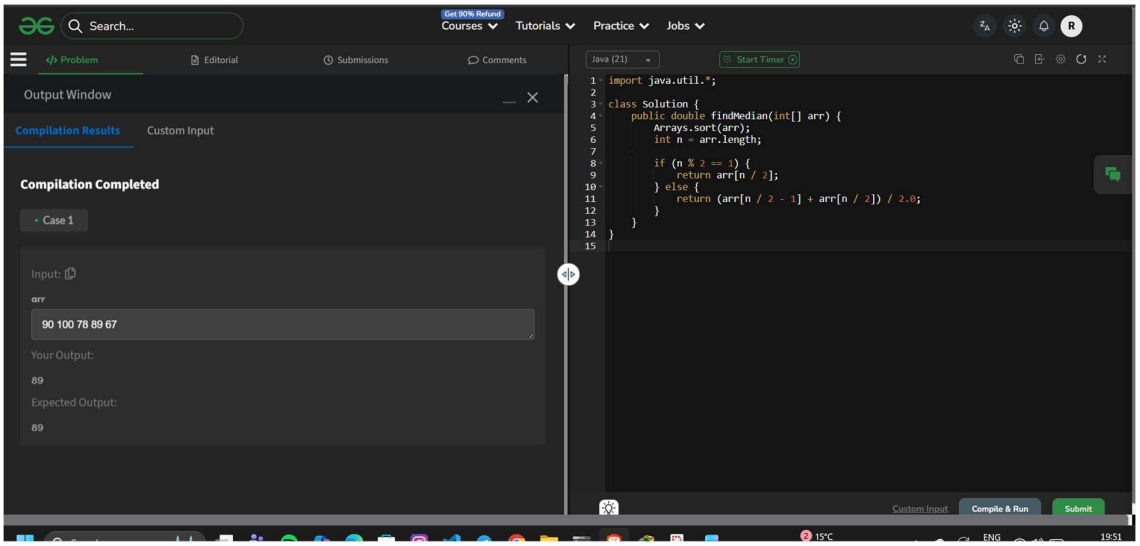
3.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
1 2 3 3 4
a =
1
b =
2
Your Output:
true
Java (21)
Start Timer
class Solution {
public void threeWayPartition(int[] arr, int a, int b) {
int low = 0, mid = 0;
int high = arr.length - 1;
while (mid <= high) {
if (arr[mid] < a) {
swap(arr, low, mid);
low++;
mid++;
}
else if (arr[mid] > b) {
swap(arr, mid, high);
high--;
}
else {
mid++;
}
}
}
private void swap(int[] arr, int i, int j) {
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
}
}
Custom Input
Compile & Run
Submit

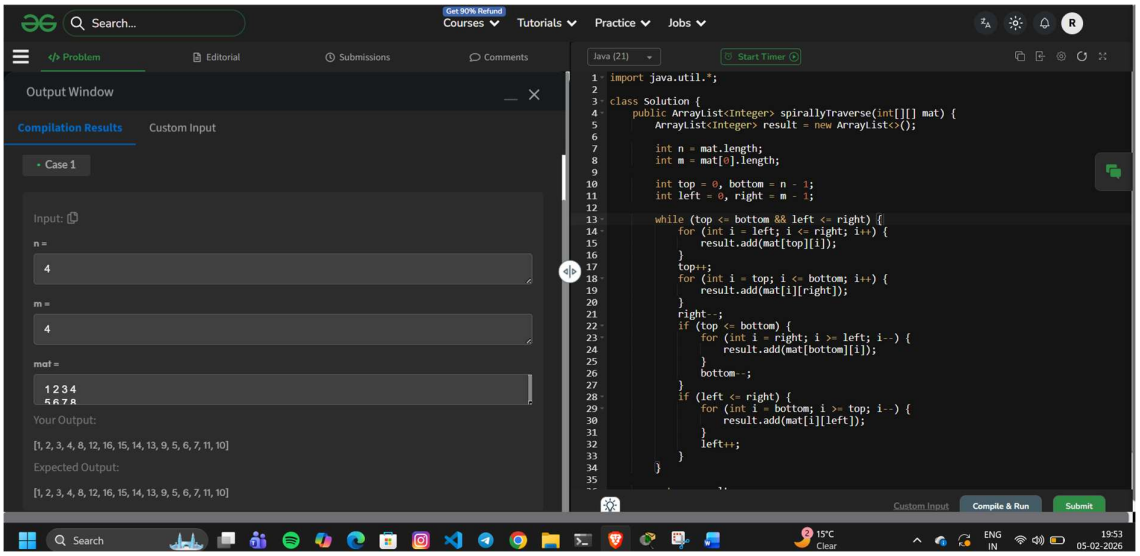
4.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[] =
2 1 5 6 3
k =
3
Your Output:
1
Expected Output:
1
Java (21)
Start Timer
class Solution {
int minSwap(int[] arr, int k) {
int n = arr.length;
int good = 0;
for (int val : arr) {
if (val <= k) good++;
}
if (good <= 1) return 0;
int bad = 0;
for (int i = 0; i < good; i++) {
if (arr[i] > k) bad++;
}
int minSwaps = bad;
for (int i = 0, j = good; j < n; i++, j++) {
if (arr[i] > k) bad--;
if (arr[j] > k) bad++;
minSwaps = Math.min(minSwaps, bad);
}
return minSwaps;
}
}
Custom Input
Compile & Run
Submit

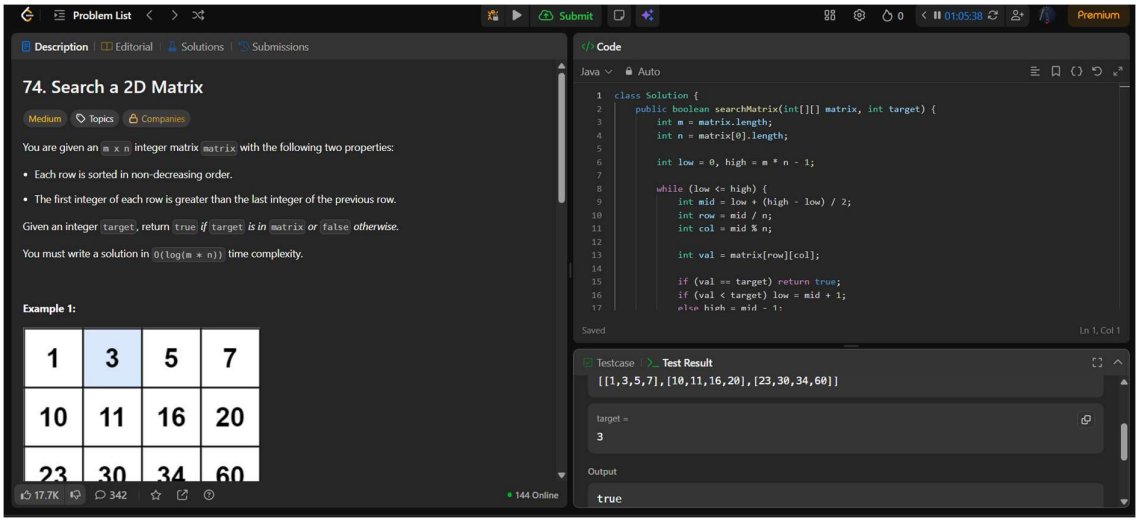
5.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr
90 100 78 89 67
Your Output:
89
Expected Output:
89
Java (21)
Start Timer
import java.util.*;
class Solution {
public double findMedian(int[] arr) {
Arrays.sort(arr);
int n = arr.length;
if (n % 2 == 1) {
return arr[n / 2];
} else {
return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
}
}
}
Custom Input
Compile & Run
Submit

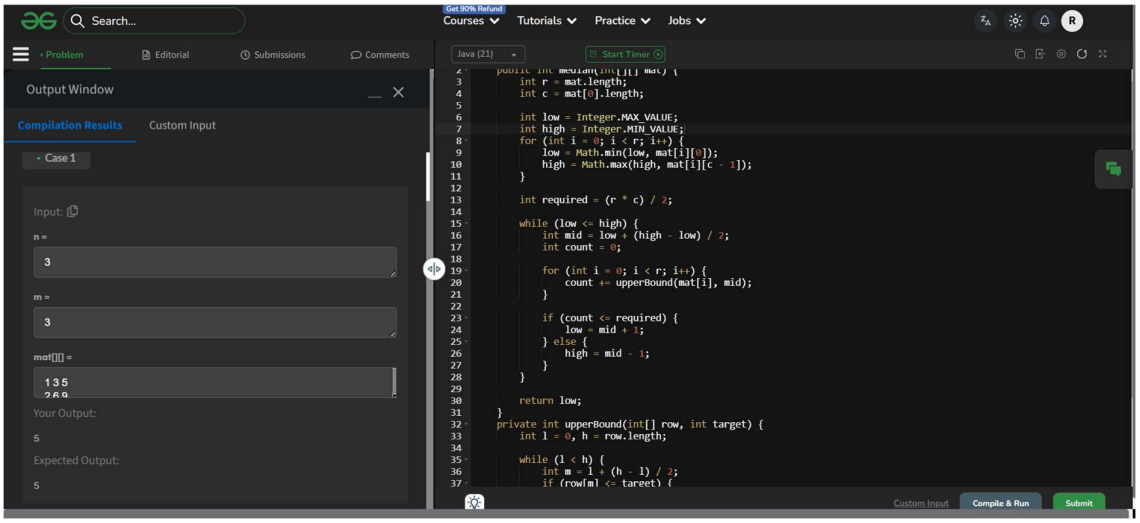
6.


Search...
Get 90% Refund
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Case 1
Input:
n =
4
m =
4
mat =
1 2 3 4
5 6 7 8
Your Output:
[1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
Expected Output:
[1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
Java (21)
Start Timer
import java.util.*;
class Solution {
public ArrayList<Integer> spirallyTraverse(int[][] mat) {
ArrayList<Integer> result = new ArrayList<>();
int n = mat.length;
int m = mat[0].length;
int top = 0, bottom = n - 1;
int left = 0, right = m - 1;
while (top <= bottom && left <= right) {
for (int i = left; i <= right; i++) {
result.add(mat[top][i]);
}
top++;
for (int i = top; i <= bottom; i++) {
result.add(mat[i][right]);
}
right--;
if (top <= bottom) {
for (int i = right; i >= left; i--) {
result.add(mat[bottom][i]);
}
bottom--;
}
if (left <= right) {
for (int i = bottom; i >= top; i--) {
result.add(mat[i][left]);
}
left++;
}
}
Custom Input
Compile & Run
Submit
Search
15°C
Clear
ENG
IN
19:53
05-02-2026

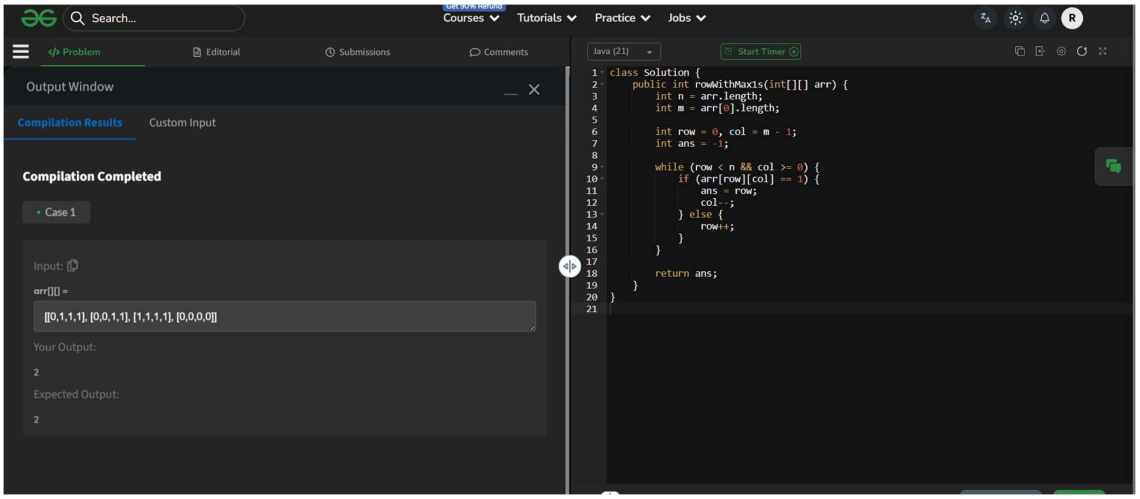
7.


Problem List
Submit
01:05:38
Premium
Description
Editorial
Solutions
Submissions
Code
74. Search a 2D Matrix
Medium
Topics
Companies
You are given an m x n integer matrix matrix with the following two properties:
Each row is sorted in non-decreasing order.
The first integer of each row is greater than the last integer of the previous row.
Given an integer target, return true if target is in matrix or false otherwise.
You must write a solution in O(log(m * n)) time complexity.
Example 1:
1 3 5 7
10 11 16 20
23 30 34 60
17.7K
342
144 Online
Java
Auto
class Solution {
public boolean searchMatrix(int[][] matrix, int target) {
int m = matrix.length;
int n = matrix[0].length;
int low = 0, high = m * n - 1;
while (low <= high) {
int mid = low + (high - low) / 2;
int row = mid / n;
int col = mid % n;
int val = matrix[row][col];
if (val == target) return true;
if (val < target) low = mid + 1;
Saved
Ln 1, Col 1
Testcase
Test Result
[[1,3,5,7],[10,11,16,20],[23,30,34,60]]
target =
3
Output
true

8.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Case 1
Input:
n =
3
m =
3
mat[][] =
1 3 5
2 6 9
Your Output:
5
Expected Output:
5
Java (21)
Start Timer
int r = mat.length;
int c = mat[0].length;
int low = Integer.MAX_VALUE;
int high = Integer.MIN_VALUE;
for (int i = 0; i < r; i++) {
low = Math.min(low, mat[i][0]);
high = Math.max(high, mat[i][c - 1]);
int required = (r * c) / 2;
while (low <= high) {
int mid = low + (high - low) / 2;
int count = 0;
for (int i = 0; i < r; i++) {
count += upperBound(mat[i], mid);
if (count <= required) {
low = mid + 1;
} else {
high = mid - 1;
return low;
private int upperBound(int[] row, int target) {
int l = 0, h = row.length;
while (l < h) {
int m = l + (h - l) / 2;
Custom Input
Compile & Run
Submit

9.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Compilation Completed
Case 1
Input:
arr[][] =
[[0,1,1,1], [0,0,1,1], [1,1,1,1], [0,0,0,0]]
Your Output:
2
Expected Output:
2
Java (21)
Start Timer
class Solution {
public int rowWithMax1s(int[][] arr) {
int n = arr.length;
int m = arr[0].length;
int row = 0, col = m - 1;
int ans = -1;
while (row < n && col >= 0) {
if (arr[row][col] == 1) {
ans = row;
col--;
} else {
row++;
return ans;